

Rubiks cube face detection and color recognition

Image Processing project

Todoruț Aurel - Dan
Group 30434

Abstract

This project implements a program that detects a single face of a Rubik's Cube in an image or a live webcam and recognizes the color of each of its nine stickers. The pipeline uses lighting normalization in the YUV color space, adaptive thresholding to find the black grid lines between stickers, connected component labeling to isolate the blobs, and a geometric filter that selects the nine blobs that best form a 3×3 grid. Each sticker is then classified into one of the six cube colors (white, yellow, red, orange, green, blue) using simple rules in HSV space, and a virtual reconstruction of the face is displayed. The low level image processing operations (morphology, connected components, convolution, Gaussian blur, adaptive threshold) were implemented from scratch during the laboratory work.

1 Description of the project

1.1 What the project is about

The goal of the project is to detect one face of a Rubik's Cube and read the colors of its 3×3 grid of stickers. The program works in two modes: it can process an image chosen by the user, or it can run on a live webcam feed and detect the face in real time. When a face is found, the program draws the bounding box of each sticker, writes the predicted color letter on top of it, and shows a virtual reconstruction of the face as a 3×3 grid of solid colors.

1.2 Definition of terms

- **Sticker:** one of the small colored squares on a face of the cube. A standard face has $3 \times 3 = 9$ stickers.
- **Face:** one of the six sides of the cube made out of 9 stickers.
- **Blob / connected component:** a group of connected pixels that are treated as a single blob after labeling.
- **Sticker candidate:** a blob that passed the size, aspect ratio and fill filters and could be a real sticker.
- **HSV:** Hue–Saturation–Value color space, more convenient than RGB for distinguishing colors.
- **YUV normalization:** a step that reduces the effect of shadows and highlights.

1.3 Input and output

Input: a color image (chosen from disk in image mode) or a frame captured from the webcam, showing one face of a Rubik's Cube facing the camera.

Output:

- the original image with the bounding box of each detected sticker and the predicted color letter (W, Y, R, O, G, B);
- a green rectangle around the whole detected face;
- a separate window with the virtual 3×3 reconstruction;
- the detected color matrix printed in the console.

1.4 Constraints on the input

The detection assumes a cooperative scene:

- only one face should be clearly visible and the face seen as an approximate square, not a strong perspective;
- the black grid lines between stickers must be visible, since the whole detection relies on them;
- lighting should not be extreme (very strong glare or near darkness break the color classification);
- the background should not contain large square colored regions similar in size to stickers, otherwise they may be picked up as candidates;
- images wider than 640 px are resized to 640 px width so that the fixed kernel sizes and area thresholds stay consistent.

2 Related work and research

The main source of inspiration for this project was the work of Justin Ng, *Automated Rubik's Cube Recognition* [1], from Stanford University. That paper identifies the full state of the cube from two photographs, each showing three faces. The author detects the faces by applying a high pass filter on the color channels to enhance the black borders, converting to grayscale and thresholding, then filtering the resulting regions by size. For color recognition the image is converted to YUV space and the chrominance is normalized with respect to intensity, after which a detector classifies each sticker into one of the six colors.

The YUV normalization idea originally comes from Kasprzak, Szynekiewicz and Czajka, *Rubik's Cube Reconstruction from Single View for Service Robot* [2], which Ng cites as well. The normalization equations used in my project are taken directly from these works:

$$\begin{aligned} Y'_p &= 128 \\ U'_p &= (U_p - 128) k_p + 128 \\ V'_p &= (V_p - 128) k_p + 128 \end{aligned} \quad k_p = \begin{cases} \frac{128}{Y_p} & \text{for } Y_p > 128 \\ \frac{256 - Y_p}{128} & \text{for } Y_p < 128 \\ 1 & \text{for } Y_p = 128 \end{cases} \quad (1)$$

My project differs from [1] in a few ways. First, I only detect a *single* face instead of reconstructing the entire cube from two views. Second, instead of a trained statistical (MAP) classifier I use a simpler method, working on the HSV hue and saturation, which is enough for one face under normal lighting. Third, I added a webcam mode, which the original paper only mentions as future work. Finally, the low level image processing operations are my own implementations from the image processing laboratory rather than library calls.

Other sources used were the OpenCV documentation¹ for image input/output and display, and the course laboratory materials for morphological operations, connected component labeling, convolution and adaptive thresholding.

¹<https://docs.opencv.org/>

3 General Approach and High Level Architecture

The program is organized as a pipeline. Each step transforms the output of the previous one until the nine stickers are isolated and classified. The high level flow is:

1. **Resize** – read a frame or an image and resize it to a maximum width of 640 px.
2. **YUV normalization** – compensate for lighting so colors are more stable (`apply_yuv_normalization`).
3. **Color conversion** – compute HSV channels for both the normalized image (used for classification) and the original image (used for edge detection).
4. **Gaussian blur** – smooth the V channel to reduce noise (`apply_gaussian_1d`).
5. **Adaptive thresholding** – extract the dark grid lines as edges (`my_adaptive_threshold`).
6. **Morphology** – erode the edges with a disk structuring element so the borders thicken and fully enclose each sticker (`erosion`).
7. **Connected components** – label every enclosed blob (`component_labeling`).
8. **Candidate filtering** – keep only blobs whose area, aspect ratio and fill ratio look like a sticker.
9. **Grid selection** – among the candidates, find the 9 that form the tightest, most uniform 3×3 cluster.
10. **Color classification** – compute the average HSV of each sticker and map it to a cube color (`classify_cube_color`).
11. **Output** – order the stickers into a 3×3 matrix, annotate the image and draw the virtual reconstruction.

The two main modules are therefore the *detection* module (steps 1–9, purely geometric/morphological) and the *recognition* module (steps 10–11, color based). The function `detect_rubiks_face` is the func that calls everything in order, and `main` handles the two input modes (webcam / image file).

Input image → Resize → YUV normalization → HSV split → Gaussian blur → Adaptive threshold → Erosion (thicken edges) → Connected components → Candidate filtering → 3×3 grid selection → Color classification → Annotated output + virtual face

4 Implementation Details

The functions up to `classify_cube_color` (morphology, labeling, convolution, Gaussian filter, adaptive threshold) come from the image processing laboratory work and were implemented by me; the cube specific logic is built on top of them.

4.1 Lighting normalization (YUV)

Shadows and highlights change the color of a sticker, which makes classification unreliable. To reduce this, the image is converted to YUV and the chrominance components U, V are scaled towards the neutral value 128 by a factor k that depends on the luminance Y (Equation (1)). Bright pixels ($Y > 128$) and dark pixels ($Y < 128$) are pulled back so that the color of a sticker stays roughly the same regardless of how it is lit. The scaled U, V are also clamped so they cannot saturate, and the luminance is set to a constant 128. The result is converted back to BGR.

```

1 double k = 1.0;
2 if (Y > 128.0) k = 128.0 / Y;
3 else if (Y < 128.0 && Y>0) k = (256.0 - Y) / 128.0;
4
5 double max_dist = max(abs(U-128.0), abs(V-128.0));
6 if (max_dist * k > 127.0) k = 127.0 / max_dist;
7
8 double U_prime = (U - 128.0) * k + 128.0;
9 double V_prime = (V - 128.0) * k + 128.0;
10 res(i, j) = Vec3b(128, clamp(U_prime), clamp(V_prime));

```

Listing 1: YUV normalization.

4.2 Gaussian blur

A Gaussian kernel of width w with $\sigma = w/6$ is built once as a 1D filter and applied twice (once horizontally, once vertically), which is the separable form of a 2D Gaussian and is much faster. Convolution is done by the generic `compute_convolution` function, which slides the kernel over the image and ignores out of bounds pixels.

4.3 Adaptive thresholding

Because lighting is uneven across the image, a single global threshold does not separate the black grid lines well. Instead, a local mean is computed with the Gaussian blur, and each pixel is compared against that local mean minus a constant C :

$$\text{edge}(i, j) = \begin{cases} 255 & \text{if } \text{src}(i, j) < \text{localMean}(i, j) - C \\ 0 & \text{otherwise} \end{cases}$$

This marks pixels that are noticeably darker than their neighborhood – i.e. the black borders between stickers.

4.4 Morphological erosion

The thresholded edges may have small gaps. A disk structuring element (built by `get_disk_strel`) is used to erode the foreground, which in this configuration thickens the dark border lines so that every sticker ends up fully enclosed by a continuous frame. This is important because the next step relies on each sticker being a separate closed region.

4.5 Connected component labeling

`component_labeling` performs a BFS fill with 8-connectivity. Every unvisited foreground pixel starts a new label, and all pixels reachable from it receive the same label. The result is an integer image where each blob has a unique id. `color_labels` assigns a random color to each label for visualization.

```

1 if (img(i,j) == 0 && labels(i,j) == 0) {
2     label++;
3     queue<pair<int,int>> q;
4     labels(i,j) = label; q.push({i,j});
5     while (!q.empty()) {
6         auto p = q.front(); q.pop();
7         for (int k = 0; k < 8; k++) {
8             int ni = p.first + di8[k];
9             int nj = p.second + dj8[k];
10            if (isInside(img,ni,nj) && img(ni,nj)==0 && labels(ni,nj)==0){
11                labels(ni,nj) = label; q.push({ni,nj});

```

```

12         }
13     }
14 }
15 }

```

Listing 2: BFS connected component labeling

4.6 Candidate filtering

For every label, the program accumulates properties in one pass: area, bounding box, center of mass, and the sum of HSV values (from both the normalized and original images). A blob becomes a sticker candidate only if:

- its area is between $\text{totalPixels}/1000$ and $\text{totalPixels}/10$ (rejects noise and background);
- its bounding box aspect ratio w/h is between 0.6 and 1.6 (stickers are roughly square);
- its fill ratio $\text{area}/(w \cdot h)$ is at least 0.5 (a real sticker fills most of its box).

4.7 Selecting the 3×3 grid

After filtering there may still be more than nine candidates. The program treats each candidate as a possible grid center: it sorts the other candidates by distance, takes the nine nearest (including itself) and scores how “grid like” they are. A group is rejected if the cluster aspect ratio is not roughly square or if it is too spread out relative to the sticker size. The remaining groups are scored by

$$\text{score} = \text{maxDist} + 100 \cdot \frac{\text{areaVariation}}{\text{meanArea}},$$

which rewards tight clusters with uniform sticker sizes. The lowest scoring group wins; if even the best score is above a threshold (600), the detection is rejected as not a cube. This makes the detector good to background clutter and is the part that lets the webcam mode reject frames with no cube.

4.8 Ordering and color classification

The nine winning stickers are sorted by row (center y) and then within each row by column (center x), giving a 3×3 reading order. For each sticker the average HSV (from the normalized image) is mapped to a cube color by simple rules:

- very low value $\rightarrow$ unknown (?);
- low saturation $\rightarrow$ white (W);
- otherwise the hue (converted to degrees) decides between red, orange, yellow, green and blue, with a saturation tie break between orange and yellow and between white and blue.

```

1 char classify_cube_color(double h_255, double s_255, double v_255) {
2     if (v_255 < 25) return '?';
3     if (s_255 < 40) return 'W';
4     double h = h_255 * 360.0 / 255.0;
5     if (h < 15.0 || h >= 330.0) return 'R';
6     if (h < 48.0) return 'O';
7     if (h < 55.0) return (s_255 > 200) ? 'O' : 'Y';
8     if (h < 85.0) return 'Y';
9     if (h < 190.0) return 'G';
10    if (h < 260.0) return (s_255 < 90) ? 'W' : 'B';
11    return 'R';

```

Listing 3: Rule based color classification.

4.9 Output and virtual reconstruction

Each sticker is drawn with a yellow bounding box and its color letter; a green rectangle marks the whole face; and a separate window shows the reconstructed face as a 3×3 grid of solid color tiles. In webcam mode an FPS counter is shown on the feed, and the detection runs on every frame.

5 User Manual

5.1 Requirements

- A C++ compiler (the project was developed in Visual Studio, hence the `stdafx.h` pre-compiled header).
- OpenCV (tested with the standard desktop build), linked to the project.
- A webcam, if the real time mode is used.

5.2 Building

1. Open the project in Visual Studio (or create a project including the source file and `common.h`).
2. Make sure the OpenCV include directory and libraries are added to the project's include/linker settings, and that the OpenCV DLLs are on the `PATH`.
3. Build in Release mode.

A minimal command line build (outside Visual Studio) looks like:

```
1 g++ main.cpp -o rubiks `pkg-config --cflags --libs opencv4`
```

5.3 Running

When the program starts it prints a menu:

```
1   Rubik's cube detector
2
3   1. Use webcam
4   2. Choose an image
5   Enter your choice (1 or 2):
```

- **Option 1 (webcam):** the program opens the camera and shows a live window. Hold a face of the cube in front of the camera; when a face is recognized the stickers are boxed and labeled and the virtual face appears. Press `ESC` or `q` to quit.
- **Option 2 (image):** a file dialog opens; pick an image. The detection windows are shown. Press any key to choose another image, or close the dialog to exit. In this mode several intermediate windows are also displayed (original, YUV normalized, Gaussian, threshold, thickened edges, labeled components) so you can see each stage.

6 Demonstration of results

The figures below show the pipeline on sample images.

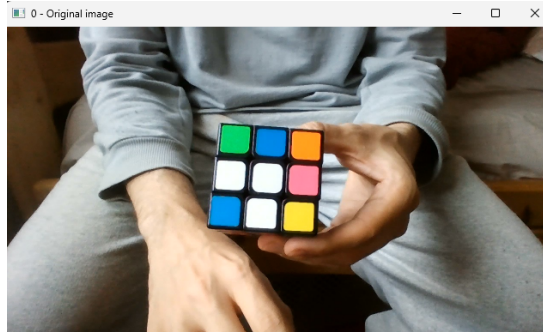


Figure 1: Original input image (resized to 640 px width).



Figure 2: Image after YUV lighting normalization.



Figure 3: Left: adaptive thresholding result (grid lines as edges). Right: edges after morphological erosion (thickened borders).



Figure 4: Connected components after labeling (each blob a random color).

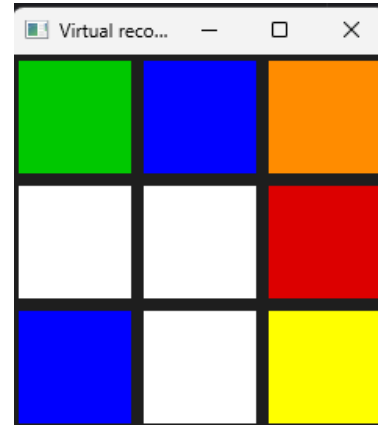
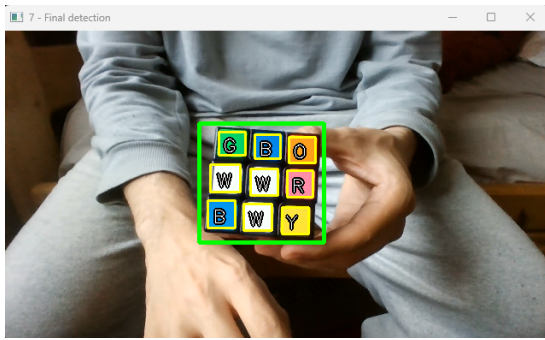


Figure 5: Left: final detection with sticker boxes, color letters and the green face box. Right: virtual 3×3 reconstruction.

The console also prints the detected face as a matrix, for example:

```

1      Final 3x3 cube face
2  W   R   G
3  O   W   Y
4  B   G   R

```

Observations. The detector works reliably when the grid lines are clearly visible and the face is roughly square to the camera. The most common failures are: (i) confusing yellow and white, orange and yellow, red and orange, since their hues are close, which is handled with a saturation tie break but can still fail under odd lighting; (ii) calling a dark blue sticker white when it is poorly lit; and (iii) picking up background squares as candidates when the background contains sticker sized colored regions. The webcam mode is more forgiving than single images because a good frame usually appears within a second or two, which matches the observation in [1] that a video feed improves the detection rate.

6.1 Results on five test images

To show some concrete results, the program was run on five frames. In each figure the original frame is on the left and the program output (the detected stickers with their color letters, inside the green face box) is on the right.

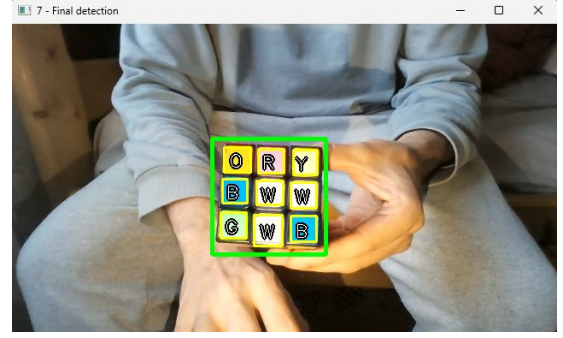
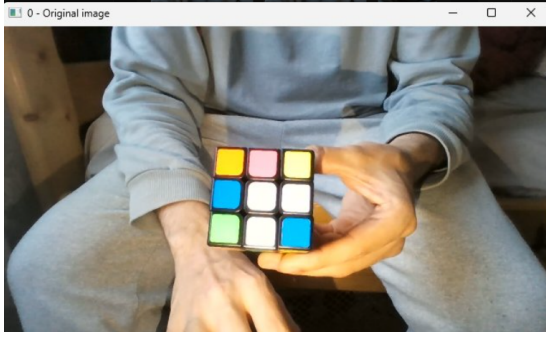


Figure 6: Test 1 – original (left) and detected face (right): O R Y / B W W / G W B.

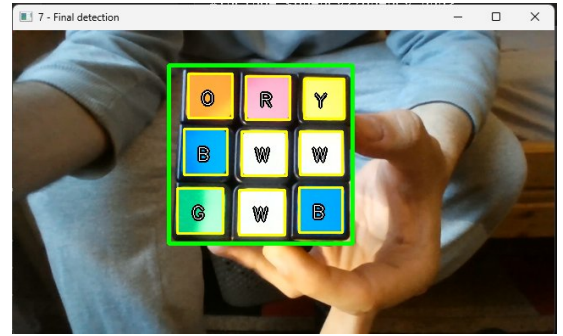
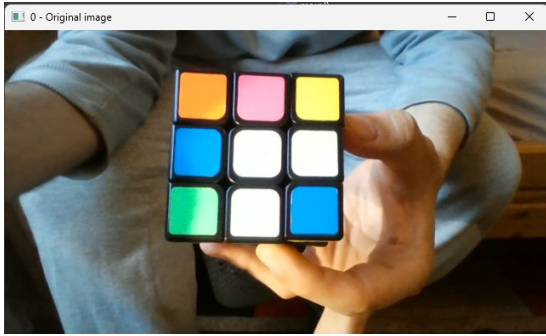


Figure 7: Test 2 – same face, different lighting. Output is identical: O R Y / B W W / G W B.

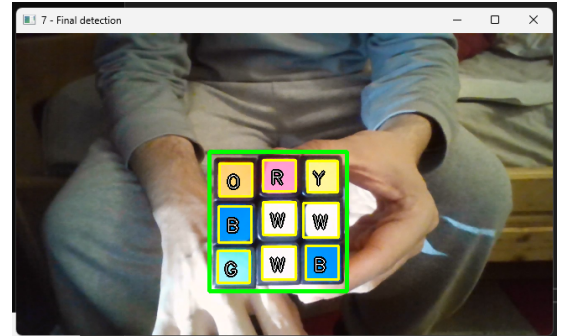
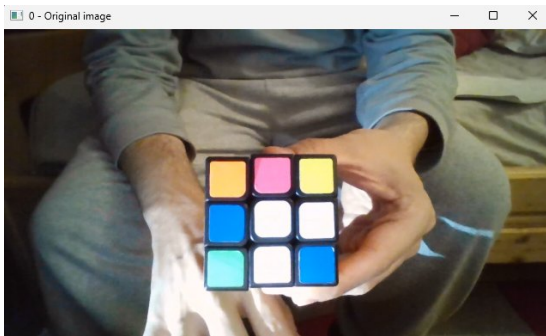


Figure 8: Test 3 – same face again, output unchanged: O R Y / B W W / G W B.

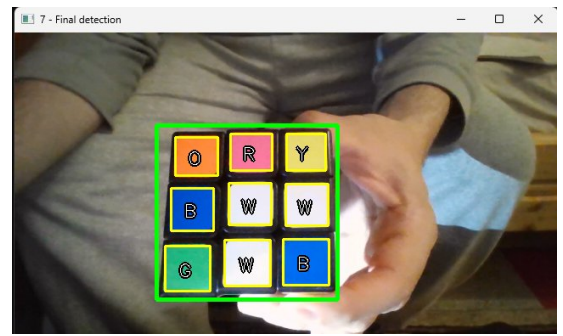


Figure 9: Test 4 – same face again, output unchanged: O R Y / B W W / G W B.

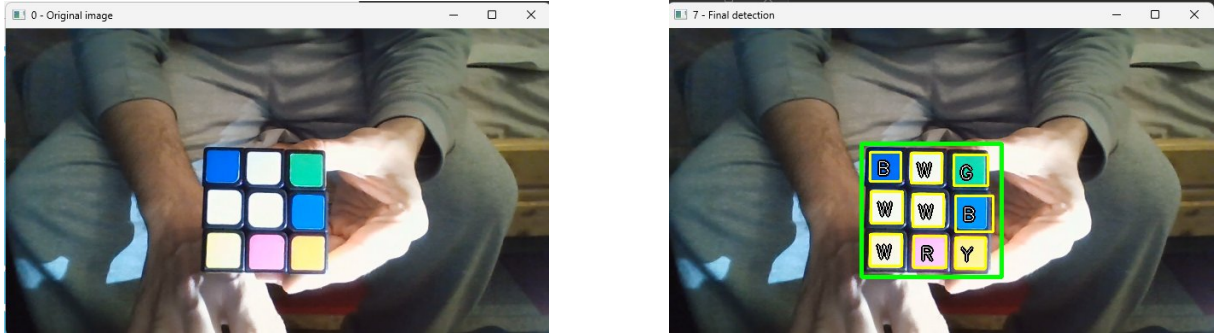


Figure 10: Test 5 – strong lighting: B W G / W W B / W R Y.

Analysis of the results. In all five frames the program located the full 3×3 grid correctly: the green box is tight around the face and exactly nine sticker boxes are drawn, so the *detection* rate is $5/5 = 100\%$. This is the part driven by the geometric grid score, and it held up across changes in lighting, distance and hand position. A useful observation is that tests 1–4 are the *same* face captured under different conditions, and the program returned exactly the same matrix every time. This repeatability is important: it shows the YUV normalization and the grid filter are stable from frame to frame, which is what makes the webcam mode usable.

For *color* recognition, comparing each output letter against the colors visible in the original frames gives the counts in Table 1. Out of 45 sticker readings, 43 matched, an accuracy of about 95.56%.

Test	Grid located	Colors correct	Note
1	9/9	9/9	pink centre read as red (nearest class)
2	9/9	9/9	identical to test 1
3	9/9	9/9	identical to test 1
4	9/9	9/9	identical to test 1
5	9/9	7/9	pale bottom left sticker read as white and orange read as yellow
Total	45/45	43/45	detection 100%, color $\approx 95.56\%$

Table 1: Per image results.

Two things are worth discussing. First, the top/centre sticker is a magenta “pink” that the classifier consistently maps to R. The cube palette has no pink class, so red is the nearest valid color, and the decision is the same in every frame; this is acceptable behaviour rather than a true error, but it shows that strongly tinted or pastel stickers sit on the edge of the hue rules. Second, the only real misclassification is in test 5, where the pale bottom left sticker is labeled W instead of yellow. This is the classic white yellow confusion: under bright light the sticker loses saturation, drops below the `s_255 < 40` threshold in `classify_cube_color`, and is therefore treated as white before the hue is even checked. Both effects point at the same limitation – the hand tuned saturation and hue thresholds – and both could be reduced by averaging the color decision over several webcam frames or by replacing the fixed rules with a small trained classifier, as in [1].

7 Conclusion

The project detects a single Rubik’s Cube face and recognizes the colors of its nine stickers, both from still images and from a live webcam. The detection is built almost entirely on classical image processing: lighting normalization in YUV, adaptive thresholding to detect the black grid, morphological thickening, connected component labeling, and a geometric filter that selects the

best 3×3 cluster of sticker shaped blobs. Color recognition is done with simple HSV rules, which proved good enough for one face under normal lighting.

References

- [1] J. Ng, *Automated Rubik's Cube Recognition*, Department of Electrical Engineering, Stanford University.
- [2] W. Kasprzak, W. Szyrkiewicz, and L. Czajka, *Rubik's Cube Reconstruction from Single View for Service Robot*.
- [3] OpenCV, *Open Source Computer Vision Library Documentation*. <https://docs.opencv.org/>